Contents

Guild Domain — Handoff Package

Date: 2026-05-29
For: Grok
Purpose: Initial context for Guild domain build

README

A working model of a human + AI team.

Guild is the operating model and intelligence layer for **mini-moi** — the connective tissue that

turns separate functional domains (Curator, Mein Deutsch, and future domains) into a coherent, learning team.

Where the other domains do specific jobs, Guild defines *how the work itself gets done*: how intent is set, labor is divided, feedback is captured, and the system gets measurably better over time.

Core Idea

A good team has roles, shared memory, and a tight loop between intent → action → real-world feedback → adjustment. Guild builds exactly that with AI agents in the team seats.

The human sets intent and makes decisions. The **Chief of Staff** carries the strategy, keeps domains healthy, scans for relevant change, and surfaces recommendations — never deciding on its own.

Documents in This Domain

- **CHARTER.md** — The vision, roles, and why Guild exists (the "what" and "why")
- **TECHNICAL_PLAN.md** — Architecture, the shared spine (Postgres + graph + temporal memory), model gateway, MCP/A2A, build roadmap, and investigation plan (the "how")

Founding Step: mini-moi Portal (minimoi.ai)

The live portal at minimoi.ai is the first visible integration point. It provides a unified, authenticated entry point to Curator and Mein Deutsch behind a clean reverse proxy + auth layer (owner / family / guest tiers).

This portal demonstrates the "Hub" concept that Guild will eventually deepen with cross-domain memory, health monitoring, and the Chief-of-Staff function.

Status

- **Phase 0** (functional domains in daily use): Complete
- **Phase 1** (Guild front door + portal as integration surface): In progress
- **Phase 2** (the spine): Next focus

Guild is part of mini-moi — not a general intelligence, a specific one.

CHARTER

(Content of guild-build-plan.md follows)

Guild

A working model of a human + AI team.

Status: Initiating — build plan · May 2026

Guild is a domain within **mini-moi**, a personal AI system. But it is a different *kind* of domain than the others, and that difference is the whole point.

Where the other domains each do a job — Curator surfaces the news and research that actually matter, Mein Deutsch teaches a language — Guild models *how the work itself gets done*: how a small team of AI agents and one human divide labor, stay pointed at a shared intent, learn from what actually happens, and get measurably better over time.

The functional domains are the *what*. Guild is the *how*.

The name is deliberate. A guild was a body of craftsmen who shared what they knew and worked together to improve their craft — each member getting better because the others did. That is the model here, with AI agents and a human as the craftsmen: a small workshop that pools what it learns and compounds it. "Memory that gets better over time" is, in those terms, just accumulated craft knowledge.

The idea

Many "AI tools" are used like vending machines: you ask, they answer, nothing persists. That is not how a good team works.

A good team has roles and division of labor. It works toward an intent set by a person, not a prompt issued in isolation. It takes feedback from the real world — what shipped, what broke, what landed — and it *remembers*, so the next attempt starts further along than the last. Over time it gets not just better but faster, because the team is tuned to the person and to the work.

Guild is an attempt to build exactly that, with AI agents in the team seats. The human is the leader: they set intent and make the calls.

Closest to the leader is a **Chief of Staff**. Like a White House chief of staff, it doesn't set the agenda — it carries it. It holds the leader's goals and forward-looking strategy, and it keeps the trains running on time across the domains: coordinating the work, sequencing what comes next, and watching that every department actually moves and stays healthy.

Carrying the strategy also means keeping it current. The craft and its tools shift constantly — a dependency is abandoned, a security hole opens, a standard converges — so the Chief of Staff scans for what's changing, flags what bears on a decision already made, advises, and recommends. What it never does is decide. The leader decides; the Chief of Staff's whole job is to make sure the leader decides *well-informed*. Scan, flag, advise, recommend — then hand the call back.

Beneath that, a standing division of labor does the building:

- **Design** — strategy, structure, and decisions before anything is built.
- **Implementation** — turning approved designs into working code.
- **Operations** — execution, file and repository work, the plumbing that keeps it running.

The agents don't replace judgment; they extend reach. The result is a team that does considerably more than a traditional one of the same size — not because any single agent is brilliant, but because the loop between intent, action, real feedback, and memory is tight and it compounds.

It is built and proven against one real person's daily life, which is what keeps it honest — it has to actually be useful, not just demonstrable. The model itself is extendable to anyone.

Where Guild sits

```
flowchart TB
    H([Human - intent and feedback])
    subgraph platform [mini-moi]
        direction LR
        C[Curator<br/>research and news]
        D[Mein Deutsch<br/>language]
        G[Guild<br/>operating model and intelligence]
    end
    S[(Shared spine<br/>Postgres + Neo4j)]
    H --> C
    H --> D
    H --> G
    C --> S
    D --> S
    G <--> S
```

The functional domains are independent — three proud, standalone systems, not a monolith. What unites them is approach and a shared goal: a system that learns *you* — why you're doing the thing, where you're going, and what you specifically struggle with.

Guild is the connective layer that makes that possible. It is where the shared memory lives and where the intelligence that reads across everything is built. In team terms, Guild is where the chief-of-staff function takes shape — the memory of what the leader wants, and the watch that keeps every domain moving and healthy.

Why now

This sequencing was deliberate. The functional domains were built first, on purpose, to generate real data and real operational experience — months of actual use, actual feedback, actual failures handled. You cannot build a learning model on top of nothing; it needs something real to learn from.

That foundation now exists. So the next phase is the one it was always pointing at: turning accumulated experience into intelligence.

The technical core: the spine

Guild is where the heavy technology focus lives. The other domains are functional; Guild is infrastructural.

The centerpiece is a shared **spine** — a persistent memory substrate spanning the whole system:

- **PostgreSQL** for the structured repositories — the records, the relations, the ground truth.

- **Neo4j** for the graph layer — the connections between things, which is where cross-domain intelligence actually comes from. A model of a person who is learning a language *for a specific place*, who follows *specific* threads in the world, and who works in a *specific* way is far more useful than three disconnected tools, and that "far more useful" lives in the edges between them.

Each domain exposes a small, fixed-shape summary of its own state; the spine is the connective tissue beneath them. Domains stay independent in code; the intelligence emerges from the shared memory, governed from Guild.

Build plan

Phase 0 — Foundation *(complete)* Functional domains in real daily use, generating the data and operational experience the rest depends on.

Phase 1 — Guild front door *(near term)* A landing page that tells this story and gives context — the document you are reading, made into a place. Look-and-feel standardized to the system the platform already uses.

Phase 2 — The spine *(next)* Stand up the PostgreSQL repositories and the Neo4j graph layer. Migrate existing domain data into structured, queryable form. This is the immediate hard-focus build.

Phase 3 — Intelligence Build the learning loops on top of the spine: remember, adjust, improve. A cross-domain model of the user that gets sharper with use.

Phase 4 — The operating model, formalized Codify the agent roles and the human-in-the-loop workflow as a reusable pattern — the extendable version of what has been proven on one life.

Access

The Guild landing — this story and its context — is open. The working surfaces behind it are private to the operator. The narrative is meant to be read; the machinery is not meant to be public.

Guild is part of mini-moi — not a general intelligence, a specific one.

TECHNICAL PLAN

Guild — Technical Plan & Stack Investigation

Engineering companion to the Guild charter.

Status: Planning · **Horizon:** June build + July–September evolution · **Landscape snapshot:** May 2026

How to read this

The charter is the *why*. This is the *how* — and, deliberately, the *what we still need to decide*. The spine is too load-bearing to lock on a default, so several choices below are left open with candidates and a decision rule, to be settled in a short pre-build investigation against real data. Where a choice is already clear, it's stated as such.

Tool and protocol facts here are a May 2026 snapshot; this space moves monthly, so treat the appendix as a dated scan, not gospel.

Principles

These shape every choice that follows.

1. **Local-first and independent.** Runs on owned hardware. No hard cloud dependency; the domains keep working without a network. Cloud is a tier, not a requirement.
2. **Model- and agent-independent.** Every model call goes through one abstraction layer. Swapping a model or running an A/B is *configuration, not code*. This is already practiced in design; it becomes architectural.
3. **Domains stay independent.** Thin adapters and summary contracts. The spine *connects* the domains; it does not absorb them.
4. **Standards over bespoke.** Adopt the converged open standards — MCP, AGENTS.md, OpenAI-compatible APIs — so the system interops and stays unlocked from any one vendor. Writing custom agent plumbing in 2026 is writing technical debt.
5. **Investigate before committing — and keep investigating after.** For load-bearing choices — the spine above all — prototype on real Curator and German data before locking. And because the landscape shifts under any locked choice, the investigation never fully ends; it becomes a standing function (see *The Chief of Staff as technology radar*).

Architecture at a glance

```
flowchart TB
    subgraph IF [Interfaces]
        TG[Telegram]
        HT[HTML / Portal / Hub]
    end
    subgraph DM [Domains - independent]
        CU[Curator]
        DE[Mein Deutsch]
        GU[Guild]
    end
    GW[Model gateway<br/>routing · A/B · cost tracking]
    subgraph MD [Models - swappable]
        AN[Anthropic]
        XA[xAI]
```

```
    LO[Local · Ollama]
end
subgraph SP [The spine]
    RE[(Relational<br/>Postgres)]
    GR[(Graph + temporal memory)]
    VE[(Vectors)]
end
IO[Interop · MCP tools / A2A agents]

IF --> DM
DM --> GW --> MD
DM --> SP
GU <--> SP
DM <--> IO
GU <--> IO
```

Four decisions define the build. Three of the four have a settled standard; the spine is the one that needs a real bake-off.

Decision 1 — The spine *(investigate before committing)*

This is the central choice. Requirements: hold structured repositories (relational), model relationships across domains (graph), support retrieval (vectors), and — critically for "remember, adjust, get better" — track **how facts change over time** (temporal). That last requirement is the one most stacks get wrong by storing only current-state snapshots.

Candidate A — Postgres-unified. PostgreSQL + Apache AGE (openCypher graph *inside* Postgres) + pgvector (vectors inside Postgres). One local store, one backup, SQL and Cypher in the same engine. Fewest moving parts, cleanest ops, fully local. Trade-off: AGE is less battle-tested for heavy graph workloads, and temporal logic (fact-validity windows) is something you'd build yourself.

Candidate B — Postgres + Neo4j + Graphiti. Relational ground truth in Postgres; the temporal knowledge graph via Graphiti (open-source) on Neo4j. Best-in-class for exactly the learning-loop goal — Graphiti stores fact-validity windows rather than timestamped snapshots, so "what did we believe in February, and why did it change" is a native query. Trade-off: two engines to run; heavier; Graphiti's graph construction can be token-expensive at write time.

Candidate C — Embedded graph. An in-process property graph with built-in vector and full-text search (RyuGraph, the maintained fork of Kuzu — note Kuzu itself was archived October 2025). Lightest possible, zero-server, maximally local. Trade-off: youngest and least proven post-fork; smallest ecosystem.

The temporal point that cuts across all three: the charter's whole thesis is a temporal-memory problem. Whatever store wins, design for temporality from day one — validity windows on facts, not just current values. Candidate B gets this for free; A and C require building it.

Build-vs-buy on memory itself: rather than build the memory engine from scratch, the accelerators in scope are Graphiti/Zep (temporal KG), Mem0 (drop-in personalization), Letta (OS-style

tiered memory), and Cognee (local-first graph RAG). A common, durable pattern is a **hybrid**: a markdown vault for canonical, human-owned, portable knowledge, plus a memory engine for transient/agent memory. The vault holds what you own; the engine holds what the agent needs to recall.

Decision rule: stand up Candidate A and Candidate B locally, load one month of real Curator + German data into each, and run 10–15 real temporal and cross-domain queries as the benchmark. Pick on measured temporal-query quality, ops burden, and cost — not on preference. Neo4j was the working default; this investigation confirms or replaces it.

Decision 2 — The model gateway *(settled: adopt a gateway; pick is swappable)*

The home of model- and agent-independence. Requirement: one internal interface; route, swap, or A/B any model (Anthropic, xAI, local) by config; keep keys and data local; track cost per call.

Primary — LiteLLM, self-hosted. OpenAI-compatible proxy across 100+ providers, with fallbacks, caching, cost tracking, and A/B routing; keys never leave the machine. Fits local-first, cost control, and A/B in one component. **Security caveat:** pin to v1.83.0 or later and verify integrity — a March 2026 PyPI supply-chain compromise hit versions 1.82.7/1.82.8. The gateway sees every key, so treat it as the most security-sensitive component in the system.

Complement — OpenRouter. Managed gateway, 300+ models, one key and one bill, low ops, roughly a 5% markup. Good for reaching breadth or running quick hosted A/Bs without self-hosting. Because both LiteLLM and OpenRouter speak the OpenAI-compatible interface, moving between them is a one-line config change — so this choice is genuinely reversible.

Local tier — Ollama. Local models as a zero-cost fallback tier; the gateway treats local and hosted models identically. Which specific local models earn their place (Gemma, Qwen, Llama, etc.) is an A/B detail *behind* the gateway, not an architecture commitment.

Design point: route by task, not by habit — cheap or local models for pre-filtering, stronger models for synthesis (Curator already does tiered scoring). The gateway makes that routing explicit and measurable, and turns "which model is better for X" from a hunch into a logged A/B.

Decision 3 — Reaching tools and other agents *(settled standards; phased adoption)*

Two layers, both now converged open standards under the Linux Foundation's Agentic AI Foundation.

MCP (Model Context Protocol) — now. The won standard for agent-to-tool access; supported by Claude, Cursor, OpenAI, and Google. This is how Guild's agents reach tools and data, and how the system exposes its own capabilities. The existing connectors formalize onto MCP. Build internal capabilities **as MCP servers**, so any agent — Claude, Cursor, or a local model — can use the same tools without reimplementation.

A2A (Agent2Agent) — horizon. The emerging standard for agent-to-agent coordination, using Agent Cards that describe what an agent can do and how to invoke it. This is the answer to "how do we reach other agentic systems / external LLM agents." Adopt when cross-agent delegation is

actually needed. A Q3 2026 MCP/A2A joint specification — the first formal protocol bridge — falls inside the evolution window below.

Sequencing matches the field consensus: MCP first (tools), A2A later (agents), with further tiers (ACP/ANP) only if decentralized discovery is ever needed.

Security: MCP best practice is read-only and scoped by default, OAuth 2.1 / PKCE, audit logging, and never exposing raw SQL or write access to untrusted agents. The spine sits *behind* MCP and is never directly reachable with write/raw access.

Decision 4 — Multi-assistant development: Claude + Cursor *(settled standard)*

Requirement: develop with both Claude Code and Cursor to balance cost and play to each tool's strengths, while keeping them reading identical rules.

AGENTS.md is the single source of truth. It is the cross-tool open standard, read natively by Cursor, Codex, Copilot, Windsurf, and others. Claude Code does not read AGENTS.md natively (as of May 2026), so bridge it: a thin `CLAUDE.md` that imports AGENTS.md (`@AGENTS.md`), or a symlink. Result: Claude and Cursor follow one set of conventions, and the existing `ORCHESTRATOR.md` aligns to or feeds AGENTS.md as the routing source of truth.

Shared capability via MCP. Tools built as MCP servers are used identically by both assistants — no per-tool divergence.

A/B the assistants, not just the models. Same task, both tools, compared — the personal A/B discipline extended from models to dev assistants. Cost-route deliberately: heavier reasoning to whichever proves better at it, bulk mechanical edits to the other.

Secrets: none of these config files hold credentials — they're committed to git. Keys live in `.env` or a secret manager, always.

The Chief of Staff as technology radar *(standing function)*

Everything above is a snapshot, and snapshots rot. Kuzu was archived; LiteLLM was compromised; a protocol bridge is landing in Q3 — none of which was visible from inside the system. A team whose premise is "remember, adjust, get better" cannot let its own toolchain quietly age while it does that for everything else. So stack-scanning is not a one-time pre-build task; it is an ongoing responsibility of the Chief of Staff — the forward-looking half of the role, alongside the domain-health watch.

The function is **scan, flag, advise, recommend — the human decides.** The radar surfaces what's changing in the craft and its tools, flags what bears on a decision already made, and recommends; it never swaps a dependency on its own. That preserves the leader-decides principle and keeps the system from churning itself.

Two things make this cheap and durable rather than a new build:

- **It is Curator pointed inward.** Curator is already a research-intelligence engine — ingestion, scoring, novelty detection, deep dives. The technology radar is that same machinery aimed at the team's own stack instead of the world. It inherits Curator's discipline: tuned

for *signal* — deprecations, security issues, standard convergence or divergence, genuine step-changes — and deliberately silent on what is merely trending. "A room to think, not a feed to scroll" applies to tooling too; chasing every new tool is its own failure mode.
- **Decisions are stored as temporal facts.** Each choice in this document lives in the spine as a fact with a validity window and its rationale ("chose X on date D, because premise P"). When the radar detects that a premise changed, it flags it *against the stored decision*: "this rested on Kuzu being maintained; it no longer is — revisit?" That is the remember/adjust/get-better loop turned on the architecture itself.

This very document was run #1 of that loop, performed by hand. The function is to run it on a cadence and route its output to the leader as recommendations.

June build plan

Phases 1–2 of the charter, made concrete. The theme is *prove the loop, not the breadth.*

- **Week 1 — Foundations.** Ship the look-and-feel cleanups (Hub + Curator standardization, specced separately). Establish `AGENTS.md` + importing `CLAUDE.md` so Claude and Cursor are aligned from here forward. Stand up the model gateway (LiteLLM, pinned v1.83.0); route existing domain model calls through it; capture a baseline cost-per-call.
- **Week 2 — Spine bake-off.** Stand up Candidate A (Postgres + AGE + pgvector) and Candidate B (Postgres + Neo4j + Graphiti) locally. Load one month of real Curator + German data into each. Write the 10–15 temporal/cross-domain benchmark queries.
- **Week 3 — Decide and migrate.** Pick the spine on the evidence. Begin migrating domain data into the chosen store *behind the summary contract* — the Hub already reads the contract, so nothing downstream changes.
- **Week 4 — First intelligence loop.** One real "remember → adjust" behavior, end to end (e.g., Curator novelty/feedback memory, or German error-pattern memory — the persistent `Mein Frau → Meine Frau` class of correction), writing to and reading from the spine. One loop working beats five half-built.

Throughout: every internal capability an agent calls gets exposed as an MCP server as it's built.

July–September evolution

- **July — Deepen the intelligence.** Build the cross-domain model of the user in the temporal graph (language goals × research interests × work intent as connected, time-aware facts). Expand the remember/adjust loops per domain. Stand up the **Chief-of-Staff function** as a real service over the spine: the intent register, the domain-health watch (which also powers the Hub's failure alerts), and the technology radar (scan/flag/advise/recommend, on a cadence) reusing Curator's pipeline pointed inward at the stack.
- **August — Agent coordination.** Introduce A2A for agent-to-agent delegation; let the Chief-of-Staff agent coordinate the domain agents through it. Begin exposing select capabilities for cross-agent use. Model and assistant routing become data-driven from gateway metrics rather than judgment calls.
- **September — Extendability and hardening.** Generalize the operating model beyond a single user — the charter's "extendable" claim made real. Align with the Q3 2026 MCP/A2A

joint spec. Add observability (OpenTelemetry across the protocol layer). Graduate Guild from `_NewDomains` into a first-class domain.

Open questions to settle in investigation

- **Spine:** A vs B vs C, decided on the real-data benchmark — not before.
- **Memory build-vs-buy:** owned markdown-vault + semantic search vs Graphiti / Mem0 / Cognee. Likely a hybrid (owned vault + one engine).
- **Chief-of-Staff split:** how much of its logic is deterministic (scheduling, health) vs model-driven (intent synthesis).
- **Local model tier:** which local models earn their place behind the gateway, decided by A/B.

Appendix — candidate tools in scope (May 2026 snapshot)

Layer	In scope	Current caveat
Relational	PostgreSQL	Stable baseline; the constant across all spine candidates
Graph	Apache AGE (on Postgres), Neo4j, RyuGraph (embedded)	Kuzu archived Oct 2025 → RyuGraph fork carries it
Vectors	pgvector	Lives in Postgres; simplest if unified
Temporal memory	Graphiti / Zep, Letta, Mem0, Cognee	Graphiti strongest on temporal; Cognee strongest local-first
Model gateway	LiteLLM (self-host), OpenRouter (managed)	LiteLLM: pin v1.83.0 after Mar 2026 supply-chain incident
Local serving	Ollama (vLLM / llama.cpp as alternatives)	Models behind the gateway are A/B details, not commitments
Tool interop	MCP	Won the tool layer; Linux Foundation AAIF
Agent interop	A2A	Emerging; Q3 2026 MCP/A2A joint spec is the first bridge
Dev config	AGENTS.md (+ CLAUDE.md import)	AGENTS.md is the cross-tool standard; Claude Code bridges via import/symlink

---

Companion to the Guild charter. Part of mini-moi — not a general intelligence, a specific one.